

claude-windows / 50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a\subagents\workflows\wf_ddb0b3fd-b40\agent-add137dd52b7ff061.jsonl`  
- SHA-256: `b4d09eba3affce16141b3b6fffed6c700d4a1591f27758c26d170a5d6f01e6e3`  
- Source modified: `2026-07-07T04:12:09+00:00`  
- Imported at: `2026-07-08T16:00:26+00:00`  
- project: `wf_ddb0b3fd-b40`  
- session_id: `50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a`
```

Transcript

1. user (2026-07-07T04:10:28.572Z)

You are reviewing an UNCOMMITTED working diff implementing issue #506 (GPU-resident WASB inference) in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. The full diff is at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch (read it). The new test file tests/test_wasb_nvdec_resident.py is untracked (not in the diff) ? read it directly from the repo.

Ground truth you must compare against:

- The VALIDATED reference implementation: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/gpu_resident/exp_t2_full.py (the GpuFrontend class + its grid build ARE what was validated on a real L4: F1 0.574 ? baseline 0.582 on the hard clip). The production port must be semantically faithful to it.
- WASB-SBDT interfaces (not in this repo; trust these facts):
TracknetV2Detector.run_tensor(imgs, affine_mats) moves imgs .to('cuda') itself and calls postprocessor.run(preds, affine_mats) which does affine_mats[scale].cpu().numpy() then indexes [i] per batch item; ImageDataset (eval) returns (imgs_t, hms_t, trans_outputs_inv, xys, visis, img_paths) where trans_outputs_inv is {scale: np.float64 [2,3]}; per-frame preprocessing is PIL RGB -> cv2.warpAffine(trans_input, (512,288), INTER_LINEAR) -> ToTensor -> Normalize(ImageNet); get_transform uses c=(w/2,h/2), s=max(h,w); OnlineTracker uses np.Inf; WASB detector.py calls torch.load(model_path) with no weights_only arg.
- The CPU decode path (run(), run_video_streaming) must be BEHAVIOR-UNCHANGED on the WSL torch-1.11 + numpy<1.24 env (byte-parity guarantee), except that torch>=2.6 envs get the weights_only + np.Inf shims.

Report ONLY defects you can ground in the actual code you read ? cite file:line. No style nits, no speculative maybes without a concrete failure scenario. Return findings as structured output.

A reviewer claims this defect in the #506 diff:

- file: backend/pipeline/detectors/native_wasb_runner.py
- line: 288
- summary: run_predict's defensive gate 'for callers that skipped healthcheck' (comment at line 304) mirrors only the nvdec+non-CUDA check, not the decode_backend validity check, so an unvalidated WASB_DECODE_BACKEND typo silently selects the cpu/--stream-video path.
- failure scenario: WASB_DECODE_BACKEND bypasses the pydantic Literal (from_indexing_cfg line 126 stores it as a plain str); a caller that invokes run_predict without healthcheck (any non-trajectory_hybrid call site, scripts, tests) with WASB_DECODE_BACKEND='nvdec-resident' (hyphen typo) falls into the 'elif self.cfg.stream_video' branch at line 293 and runs the cv2 streaming path with a cpu-keyed cache ? the operator believes the GPU-resident backend ran, and nothing errors or warns.
- severity: minor

Adversarially VERIFY it: read the actual code (and the validated reference / WASB interface facts above) and try to REFUTE the claim. It is only real if the concrete failure scenario actually happens in the code as written. Default to isReal=false if the scenario cannot occur,

is already guarded, or rests on a misreading.

2. user (2026-07-07T04:10:39.005Z)

```
80         )
81         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
82         sport: str = "badminton"
83         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
84         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
85         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
86         keep_frames: bool = (
87             False # retain the decoded frame cache after success (debug only)
88         )
89         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
90         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
91         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
92         stream_video: bool = True
93         # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
94         # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
95         batch_size: int = 0
96         prefetch_batches: int = 0
97         # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend /
WASB_DECODE_BACKEND).
98         # "cpu" (default) = omit the flag ? wasb_infer.py's historical cv2 decode paths,
argv
99         # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
only).
100         decode_backend: str = "cpu"
101
102     @classmethod
103     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
104         if isinstance(idx_cfg, dict):
105             idx_cfg = IndexingConfig(**idx_cfg)
106         w = idx_cfg.wasb
107         env = os.environ.get
108         # stream_video is tri-state in config: None/absent => native default (stream);
an
109         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
110         sv = w.stream_video
111         return cls(
112             repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
113             python_bin=env("WASB_PYTHON") or w.python_bin,
114             weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
115             sport=w.sport,
116             device=env("WASB_DEVICE") or w.device,
117             scratch_dir=env("WASB_SCRATCH")
118             or env("RALLY_SCRATCH")
119             or env("TMPDIR")
120             or "",
121             timeout_sec=w.timeout_sec,
122             keep_frames=w.keep_frames,
123             stream_video=(True if sv is None else bool(sv)),
124             batch_size=int(w.batch_size or 0),
125             prefetch_batches=int(w.prefetch_batches or 0),
126             decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
127         )
128
129
130     class NativeWasbRunner(DetectorRunner):
131         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
```

```

132
133     def __init__(self, cfg: NativeWasbConfig):
134         self.cfg = cfg
135         # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
136         # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
137         self._cuda_available: Optional[bool] = None
138
139         # --- low-level native exec (the single place tests patch)
-----
140     def _run(
141         self, argv: List[str], cwd: Optional[str] = None
142     ) -> subprocess.CompletedProcess:
143         logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
144         return subprocess.run(
145             argv,
146             cwd=cwd,
147             capture_output=True,
148             text=True,
149             timeout=(self.cfg.timeout_sec or None),
150         )
151
152         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
153     def _probe_cuda(self) -> bool:
154         """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
155         (missing python / timeout / import error) is treated as 'no CUDA'."""
156         try:
157             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
158         except (FileNotFoundError, subprocess.TimeoutExpired):
159             return False
160         return res.returncode == 0 and "cuda True" in (res.stdout or "")
161
162     def _cuda_is_available(self) -> bool:
163         """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
164         if self._cuda_available is None:
165             self._cuda_available = self._probe_cuda()
166         return self._cuda_available
167
168     def _effective_device(self) -> str:
169         """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
170         probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
171         if self.cfg.device == AUTO:
172             return DeviceContext(AUTO, self._cuda_is_available()).effective
173         return self.cfg.device
174
175     def _repo_src(self) -> str:
176         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
177
178     def _copy_infer_script(self) -> bool:
179         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
180         repo_src = self._repo_src()
181         try:
182             os.makedirs(repo_src, exist_ok=True)
183             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
184             return True
185         except OSError as e:
186             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
187             return False

```

```

188
189     def _rm_rf(self, path: Optional[str]) -> None:
190         """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
191         if path:
192             shutil.rmtree(path, ignore_errors=True)
193
194     def healthcheck(self) -> Tuple[bool, str]:
195         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
196         if not self.cfg.weights_path:
197             return False, (
198                 "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).",
199             )
200         weights = os.path.expanduser(self.cfg.weights_path)
201         if not os.path.exists(weights):
202             return False, f"WASB weights not found at {weights}"
203         repo_src = self._repo_src()
204         if not os.path.isdir(repo_src):
205             return False, (
206                 f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
207                 f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).",
208             )
209         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
210         # argparse rc=2 from wasb_infer.py at run time.
211         if self.cfg.device not in VALID_DEVICES:
212             return False, (
213                 f"unknown device {self.cfg.device!r} ? valid: {'',
'.join(VALID_DEVICES)}.",
214             )
215         if self.cfg.decode_backend not in _VALID_DECODE_BACKENDS:
216             return False, (
217                 f"unknown decode_backend {self.cfg.decode_backend!r} ? valid: "
218                 f"{'', '.join(_VALID_DECODE_BACKENDS)}.",
219             )
220         try:
221             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
222         except FileNotFoundError:
223             return (
224                 False,
225                 f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
226             )
227         except subprocess.TimeoutExpired:
228             return False, "native torch healthcheck timed out."
229         out = (res.stdout or "").strip()
230         if res.returncode != 0:
231             return (
232                 False,
233                 f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}",
234             )
235         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
236         self._cuda_available = "cuda True" in out
237         ctx = DeviceContext(self.cfg.device, self._cuda_available)
238         if ctx.cuda_required_but_missing:
239             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
240             # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
241             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
242         if ctx.fell_back:
243             logger.warning(
244                 "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "

```

```

245         "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
246         out,
247     )
248     # NVDEC decode is CUDA-only: an auto?cpu fallback (or explicit cpu/mps, which
the
249     # config validator already rejects) cannot run the GPU-resident path ? fail loud
250     # instead of letting wasb_infer.py die mid-run on a paid box.
251     if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
252         return False, (
253             f"decode_backend=nvdec_resident requires CUDA, but the effective device
is "
254             f"{ctx.effective!r} ({out}) ? NVDEC decode runs on the GPU."
255         )
256     return True, out
257
258 def run_predict(
259     self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
260 ) -> Optional[str]:
261     """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
262
263     Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
264     runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
265     same downstream contract, so the parity comparison is apples-to-apples.
266     """
267     output_dir = os.path.abspath(output_win_dir)
268     os.makedirs(output_dir, exist_ok=True)
269     # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
270     norm = str(video_win_path).replace("\\", "/")
271     stem, _ext = os.path.splitext(os.path.basename(norm))
272     # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
273     key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
274     out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
275
276     if not self._copy_infer_script():
277         logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
278         return None
279
280     weights = os.path.expanduser(self.cfg.weights_path)
281     argv = [
282         self.cfg.python_bin,
283         "wasb_infer.py",
284         "--video",
285         os.path.abspath(str(video_win_path)),
286     ]
287     frames_dir: Optional[str] = None
288     if self.cfg.decode_backend == "nvdec_resident":
289         # GPU-resident path (#506): wasb_infer.py decodes the video on the GPU
itself
290         # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so
neither
291         # --stream-video nor --frames_out_dir applies and there is no frame cache to
clean.
292         argv += ["--decode-backend", self.cfg.decode_backend]
293     elif self.cfg.stream_video:
294         # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
295         argv.append("--stream-video")
296     else:
297         scratch = self.cfg.scratch_dir or output_dir
298         frames_dir = os.path.join(scratch, f"{key}_frames")

```

```

299         argv += ["--frames_out_dir", frames_dir]
300     # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
301     # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
302     device = self._effective_device()
303     if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
304         # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu on
a
305         # GPU-less box): NVDEC decode cannot run without CUDA ? fail before the
subprocess.
306         logger.error(
307             "decode_backend=nvdec_resident requires CUDA, but the effective device
is "
308             "%r ? aborting before the WASB subprocess.",
309             device,
310         )
311     return None
312     argv += [
313         "--weights",
314         weights,
315         "--sport",
316         self.cfg.sport,
317         "--device",
318         device,
319         "--out",
320         out_csv,
321     ]
322     # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
323     # byte-parity guarantee) is unchanged when the knobs are unset.
324     if self.cfg.batch_size:
325         argv += ["--batch-size", str(self.cfg.batch_size)]
326     if self.cfg.prefetch_batches:
327         argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
328
329     logger.info(
330         "Running native WASB inference (device=%s%s) on %s ...",
331         device,
332         " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
333         os.path.basename(norm),
334     )
335     try:
336         res = self._run(argv, cwd=self._repo_src())
337     except FileNotFoundError:
338         logger.error(
339             "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin

```

3. user (2026-07-07T04:10:42.022Z)

```

9:+     # GPU-RESIDENT DECODE (#506) ? DEFAULT OFF. "nvdec_resident" moves
decode+resize+normalize
18:+     decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
21:+     def _nvdec_resident_requires_cuda(self) -> "WasbIndexConfig":
24:+         # healthcheck hard-fails if auto resolves to cpu with this backend on.
25:+         if self.decode_backend == "nvdec_resident" and self.device not in ("cuda", "auto"):
27:+             f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda' or "
43:+# healthcheck rejects anything else up front so a typo can't reach argparse as rc=2 mid-run.
44:+_VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")
53:+     # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend / WASB_DECODE_BACKEND).
55:+     # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
only).
56:+     decode_backend: str = "cpu"
64:+     decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
72:+     if self.cfg.decode_backend not in _VALID_DECODE_BACKENDS:

```

```

74:+         f"unknown decode_backend {self.cfg.decode_backend!r} ? valid: "
87:+         if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
89:+             f"decode_backend=nvdec_resident requires CUDA, but the effective device is "
94:     def run_predict(
100:+         if self.cfg.decode_backend == "nvdec_resident":
104:+             argv += ["--decode-backend", self.cfg.decode_backend]
113:+             if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
114:+                 # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu on a
117:+                 "decode_backend=nvdec_resident requires CUDA, but the effective device is "
138:+`--decode-backend nvdec_resident` (default `cpu`) moves decode + resize + normalize
152:+# --stream-video); "nvdec_resident" = GPU-resident NVDEC decode + affine grid_sample
154:+DECODE_BACKENDS = ("cpu", "nvdec_resident")
217:+     decode_backend: str = "cpu",
225:+     ``decode_backend`` follows the same pattern (#506): pre-existing caches (no field)
226:+     are ``cpu``-decoded, so they stay compatible with cpu runs ? but an ``nvdec_resident``
237:+     and manifest.get("decode_backend", "cpu") == decode_backend
264:+     decode_backend="cpu", # run() IS the cpu decode path (disk frames / cv2 stream)
272:+     "decode_backend": "cpu",
378:+     ""On-GPU frame source for the ``nvdec_resident`` detector pass (#506).
401:+         f"decode_backend=nvdec_resident needs PyNvVideoCodec with RGB output "
403:+         f"/ gpu_setup.sh stage them). Fall back to decode_backend=cpu."
446:+def run_video_nvdec_resident(
466:+     ``decode_backend`` and never shared with cpu-decoded runs. Same resumable
478:+     raise SystemExit("decode_backend=nvdec_resident requires a CUDA GPU")
493:+     f"nvdec_resident: {total} frames ({fe.orig_w}x{fe.orig_h}), "
497:+     # ---- cache setup / resume decision (mirrors run()); keyed nvdec_resident) --- #
512:+     decode_backend="nvdec_resident",
533:+     "decode_backend": "nvdec_resident",
562:+     f"(total {windows_total}, batch={batch_size}, source=nvdec_resident)..."
642:+     "--stream-video). nvdec_resident = GPU-RESIDENT decode+resize+normalize "
654:+     if args.decode_backend == "nvdec_resident":
658:+         raise SystemExit("--decode-backend nvdec_resident requires --video")
661:+         "--decode-backend nvdec_resident requires --device cuda "
664:+         run_video_nvdec_resident(
726:+# PyNvVideoCodec dlopens. Harmless when decode_backend=cpu; required for nvdec_resident.
744:+# not load) for decode_backend=nvdec_resident. wasb_infer.py carries the two version shims
this bump
780:+# PyNvVideoCodec for the GPU-resident decode backend (indexing.wasb.decode_backend=
781:+# nvdec_resident, ~1.7x). Step [2/6] stages the NVDEC/NVENC driver userspace libs
871:+except Exception as e: # nvdec_resident unavailable; the cpu decode path still works
925:+     # inject neither) ? a silent drop of any of these bricks decode_backend=nvdec_resident.
942:+def test_wasb_decode_backend_default_off_and_reaches_native_runner():
943:+     ""#506: decode_backend defaults to "cpu" (the flag is default-OFF until the full
949:+     assert WasbIndexConfig().decode_backend == "cpu"
951:+     NativeWasbConfig.from_indexing_cfg(AppConfig().indexing).decode_backend
954:+     idx = AppConfig(indexing={"wasb": {"decode_backend": "nvdec_resident"}}).indexing
955:+     assert NativeWasbConfig.from_indexing_cfg(idx).decode_backend == "nvdec_resident"
958:+def test_wasb_decode_backend_env_override_beats_config(monkeypatch):
961:+     monkeypatch.setenv("WASB_DECODE_BACKEND", "nvdec_resident")
963:+     assert cfg.decode_backend == "nvdec_resident"
966:+def test_wasb_decode_backend_rejects_typos_and_non_cuda_device():
967:+     ""A typo'd backend and the contradictory nvdec+cpu-device combo must fail at
969:+     native runner's healthcheck hard-fails if auto resolves to cpu.""
975:+     WasbIndexConfig(decode_backend="nvdec") # Literal rejects unknown backends
977:+     WasbIndexConfig(decode_backend="nvdec_resident", device="cpu")
981:+     "wasb": {"decode_backend": "nvdec_resident", "device": "mps"}
986:+     WasbIndexConfig(decode_backend="nvdec_resident").decode_backend
987:+     == "nvdec_resident"
990:+     WasbIndexConfig(decode_backend="nvdec_resident", device="auto").device
995:+def test_cloud_serving_preset_keeps_decode_backend_default_off():
997:+     nvdec_resident until the full golden set passes at the SERVED preset ? a
1001:+     assert "decode_backend" not in preset_for("cloud-serving")["indexing"]["wasb"]
1016:+def test_run_predict_omits_decode_backend_flag_by_default(tmp_path):
1017:+     ""decode_backend defaults to "cpu" ? the flag is NOT passed, so the default argv
1025:+def test_run_predict_nvdec_resident_threads_flag_and_skips_cv2_paths(tmp_path):

```

```

1026:+     """nvdec_resident decodes on the GPU inside wasb_infer.py: the flag is threaded and
1030:+         NativeWasbConfig(weights_path="/m/w.pth", decode_backend="nvdec_resident"),
1034:+     assert argv[argv.index("--decode-backend") + 1] == "nvdec_resident"
1040:+ def test_run_predict_nvdec_resident_auto_without_gpu_aborts_before_subprocess(tmp_path):
1041:+     """device=auto resolving to cpu cannot host NVDEC decode: run_predict fails BEFORE
1045:+         weights_path="/m/w.pth", device="auto", decode_backend="nvdec_resident"
1051:+         out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
1056:+ def test_healthcheck_rejects_unknown_decode_backend(tmp_path):
1057:+     """A typo'd decode_backend fails fast with a clear message, before any subprocess."""
1058:+     r = NativeWasbRunner(_ok_cfg(tmp_path, decode_backend="nvdec"))
1060:+     ok, msg = r.healthcheck()
1061:+     assert not ok and "decode_backend" in msg
1065:+ def test_healthcheck_nvdec_resident_requires_effective_cuda(tmp_path):
1066:+     """auto?cpu fallback is fine for the cpu decode path but NOT for nvdec_resident:
1067:+     the healthcheck must hard-fail instead of warn-and-proceed."""
1069:+     _ok_cfg(tmp_path, device="auto", decode_backend="nvdec_resident")
1074:+     ok, msg = r.healthcheck()
1075:+     assert not ok and "nvdec_resident" in msg
1078:+ def test_healthcheck_nvdec_resident_ok_with_cuda(tmp_path):
1080:+     _ok_cfg(tmp_path, device="cuda", decode_backend="nvdec_resident")
1085:+     ok, _msg = r.healthcheck()

```

4. assistant (2026-07-07T04:10:55.457Z)

Let me verify the config class type and the call sites.

5. user (2026-07-07T04:10:57.342Z)

```

1     """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3     The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4     "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5     It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6     Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7     that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8     see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10    * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11    ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12    (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13    * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14    (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15    strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16    * weights / repo / python / scratch come from the environment first
17    (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18    ``indexing.wasb`` config, then a default ? so a box needs no config edits.
19
20    It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21    name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22    ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23    the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24    (``trajectory_hybrid.resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26    Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
runner
27    (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28    candidate windows within a small float tolerance (~1e-3 px). cuDNN is non-

```



```

deterministic
29     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31     vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32     artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34     not the numbers.
35     """
36
37     import logging
38     import os
39     import shutil
40     import subprocess
41     from dataclasses import dataclass
42     from typing import List, Optional, Tuple
43
44     from backend.config.models import IndexingConfig
45     from backend.pipeline.detectors.base import DetectorRunner
46     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51     logger = logging.getLogger(__name__)
52
53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/data loaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (
60         "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61     )
62
63     # Decode backends wasb_infer.py understands (#506). Mirrored by its --decode-backend
choices;
64     # healthcheck rejects anything else up front so a typo can't reach argparse as rc=2 mid-
run.
65     _VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")
66
67
68     @dataclass
69     class NativeWasbConfig:
70         """Resolved settings for a Linux-native WASB run.
71
72         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
73         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
74         ``device``
75         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
76         """
77         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
78         python_bin: str = (
79             "python" # the `wasb` conda env python (invoked by path, no activate)
80         )

```

```

backend\worker\_main__.py:593:         traj = runner.run_predict(local_video, traj_dir,
video_id=video_id)
tests\test_detector_keying.py:36:         result = runner.run_predict(r"C:\folderA\match.mp4",
out_dir, video_id=video_id)
tests\test_detector_keying.py:67:         result = r.run_predict(r"C:\v\match.mp4", out_dir)
tests\test_detector_keying.py:79:         result = r.run_predict(
backend\pipeline\detectors\wasb_runner.py:130:     def run_predict(
backend\pipeline\detectors\tracknet_runner.py:131:     def run_predict(
backend\pipeline\detectors\stub_runner.py:111:     def run_predict(
backend\pipeline\detectors\base.py:15:- run_predict(video_win_path: str, output_win_dir: str) ->
Optional[str]
backend\pipeline\detectors\base.py:204:     def run_predict(
backend\pipeline\detectors\native_wasb_runner.py:258:     def run_predict(
backend\pipeline\segmenters\trajectory_hybrid.py:299:         csv_path =
runner.run_predict(video_path, out_dir, video_id=video_id)
docs\CODE_MAP.md:83: -> WasbRunner.run_predict(video, out_dir)
docs\CODE_MAP.md:154:[Omitted long matching line]
docs\COMPUTE_DECOUPLED_SERVING\README.md:68:1. **DETECT (GPU):**
`DetectorRunner.run_predict(video_path, output_dir, video_id) ? CSV[Frame,Visibility,X,Y]`
(`backend/pipeline/detectors/base.py:164`). Three interchangeable impls ? `WasbRunner` (WSL),
`NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ? emit the **identical CSV
schema**. The trajectory CSV is the stable artifact boundary.
tests\test_native_wasb_runner.py:220:def
test_healthcheck_caches_probe_for_run_predict(tmp_path):
tests\test_native_wasb_runner.py:256:         out = r.run_predict(r"C:\v\clip.mp4",
str(tmp_path), video_id="abc123abc123")
tests\test_native_wasb_runner.py:285:         out = r.run_predict(r"C:\v\clip.mp4",
str(tmp_path), video_id=vid)
tests\test_native_wasb_runner.py:412:         r.run_predict(r"C:\v\clip.mp4", str(tmp_path),
video_id="abc123abc123")
tests\test_native_wasb_runner.py:423:         r.run_predict(r"C:\v\clip.mp4", str(tmp_path),
video_id="abc123abc123")
tests\test_native_wasb_runner.py:435:         r.run_predict(r"C:\v\clip.mp4", str(tmp_path),
video_id="abc123abc123")
tests\test_native_wasb_runner.py:448:         r.run_predict(r"C:\v\clip.mp4", str(tmp_path),
video_id="abc123abc123")
tests\test_native_wasb_runner.py:497:         out = r.run_predict(r"C:\v\clip.mp4",
str(tmp_path), video_id="abc123abc123")
tests\test_stub_runner.py:30:         csv_path = runner.run_predict(
tests\test_stub_runner.py:43:         csv_path = runner.run_predict("clip.mp4", str(tmp_path),
video_id="deadbeefcafe00")
tests\test_stub_runner.py:60:         a = StubDetectorRunner().run_predict("x.mp4", str(tmp_path /
"a"), video_id="v1")
tests\test_stub_runner.py:61:         b = StubDetectorRunner().run_predict("x.mp4", str(tmp_path /
"b"), video_id="v1")
tests\test_stub_runner.py:69:         out = runner.run_predict("any.mp4", str(tmp_path / "out"),
video_id="zz")
tests\test_tracknet_runner.py:456:         assert r.run_predict(r"C:\v\clip.mp4", "out") is None
tests\test_tracknet_runner.py:461:         assert r.run_predict(r"C:\v\clip.mkv", str(tmp_path)) is
None
tests\test_tracknet_runner.py:473:         result = r.run_predict(r"C:\v\clip.mp4", out_dir)
tests\test_tracknet_runner.py:480:         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path))
is None
tests\test_tracknet_runner.py:487:         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path))
is None
tests\test_tracknet_runner.py:498:         assert r.run_predict(str(src), str(tmp_path)) is None
tests\test_tracknet_runner.py:563:         result = r.run_predict(r"C:\v\clip.mp4", out_dir)
tests\test_tracknet_runner.py:617:         result = r.run_predict(r"C:\v\clip.mp4", out_dir,
video_id="abc123abc123def")
tests\test_wsl_tilde_quoting.py:103:         out = r.run_predict("C:\\vids\\clip.mp4", "out",
video_id="abcdefabcdef0000")
tests\test_wsl_tilde_quoting.py:132:         out = r.run_predict(
tests\test_worker.py:608:         def run_predict(self, video, out_dir, video_id=None):
tests\test_wasb_runner.py:64:         r.run_predict(r"C:\v\clip.mp4", str(tmp_path),
video_id="abc123abc123")

```

```

tests\test_wasb_runner.py:118:         out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path),
video_id=vid)
tests\test_wasb_runner.py:190:         out = r.run_predict(str(src), str(tmp_path),
video_id="abc123abc123")
tests\test_wasb_runner.py:206:         r.run_predict(str(src), str(tmp_path),
video_id="abc123abc123")

```

7. assistant (2026-07-07T04:11:11.323Z)

`NativeWasbConfig` is a plain `@dataclass` (line 68), so the env-sourced string is unvalidated there. Now let me check whether the production call sites actually skip healthcheck.

8. user (2026-07-07T04:11:12.527Z)

```

540         if u.lower().endswith(".csv")
541     )
542     if len(label_uris) < 2:
543         print(
544             f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
545             f"under {corpus}/labels/"
546         )
547         return 2
548
549     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
550     runner = None
551     video_by_stem: dict = {}
552     if args.trajectory_source == "detector":
553         os.makedirs(videos_dir, exist_ok=True)
554         runner = _resolve_train_detector(args, config)
555         if runner is None:
556             return 2
557         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
558             vname = storage.parse_uri(vuri).name
559             if not vname.lower().endswith(_VIDEO_EXTS):
560                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
561             video_by_stem[os.path.splitext(vname)[0]] = vuri
562
563     print(f"[worker] trajectory source: {args.trajectory_source}")
564     specs: List[dict] = []
565     for uri in label_uris:
566         fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
567         name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
568         local_label = storage.fetch(
569             uri, os.path.join(labels_dir, fname), config=storage_cfg
570         )
571         if args.trajectory_source == "detector":
572             vuri = video_by_stem.get(name)
573             if not vuri:
574                 print(
575                     f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
576                 )
577                 continue
578             local_video = storage.fetch(
579                 vuri,
580                 os.path.join(videos_dir, storage.parse_uri(vuri).name),
581                 config=storage_cfg,
582             )
583             video_id = compute_video_id(local_video)
584             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
585             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.

```

```

586         meta = _probe_video_fps_width(local_video)
587         if meta is None:
588             print(
589                 f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
590             )
591             continue
592         fps, frame_width = meta
593         traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
594         if not traj:
595             print(
596                 f"[worker] detector produced no trajectory for {name!r} ? skipping."
597             )
598             continue
599         specs.append(
600             {
601                 "name": name,
602                 "trajectory": traj,
603                 "labels": local_label,
604                 "fps": fps,
605                 "frame_width": frame_width,
606             }
607         )
608     else:
609         from backend.pipeline.detectors.stub_runner import (
610             synth_trajectory_from_labels,
611         )
612
613         traj = os.path.join(traj_dir, f"{name}_traj.csv")
614         synth_trajectory_from_labels(
615             local_label, traj, fps=args.fps, frame_width=args.frame_width
616         )
617         specs.append(
618             {
619                 "name": name,

```

9. user (2026-07-07T04:11:13.657Z)

backend\main.py:381: # real GPU box) and importing it would add seconds to this probe. The detector healthcheck is the

backend\main.py:1238: the server stays torch-light; the detector healthcheck is the GPU authority. Extracted so the

backend\results.py:6:(e.g. missing API key, WSL healthcheck failure, provider error, etc.).

backend\config\models.py:137: # healthcheck hard-fails if auto resolves to cpu with this backend on.

backend\reporting\spec.yaml:64: # provider error / quota / WSL healthcheck issue rather than a truly empty video.

backend\reporting\spec.yaml:74: failed WSL healthcheck ? these all return [] silently. Re-run with -u for

backend\config\presets.py:111: treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a hard

backend\config\startup.py:13: own healthcheck is the real authority. A warning surfaces it early without false-failing.

backend\config\startup.py:216: # GPU heads-up (warning only ? the detector healthcheck is the authority; the probe is

backend\config\startup.py:225: "has no GPU the detector healthcheck will hard-fail ? set indexing.wasb.device="

backend\config\startup.py:236: "can be a false alarm; the runner's DeviceContext/healthcheck is the authority.",

backend\worker__main__.py:502: ok, msg = runner.healthcheck()

backend\pipeline\segmenters\base.py:56: "no segments found" from "something went wrong" (missing key, healthcheck

backend\pipeline\detectors\base.py:11:- Provide a single place to evolve the healthcheck / error contract and thread

backend\pipeline\detectors\base.py:21:- healthcheck() -> tuple[bool, str]

```

backend\pipeline\detectors\base.py:31:2. Implement run_predict and healthcheck (return (bool,
str) tuple).
backend\pipeline\detectors\base.py:41:6. Add unit tests that exercise the contract (isinstance,
healthcheck shape, run
backend\pipeline\detectors\base.py:51:- healthcheck can just check "weights file exists and
torch importable + CUDA?".
backend\pipeline\detectors\base.py:57:The healthcheck message is already logged by the hybrids
on failure. The reporting
backend\pipeline\detectors\base.py:216:    def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\device.py:9:healthcheck hard-failed on a box with no GPU, so a CPU-
only Linux box (cpu-serving / a
backend\pipeline\segmenters\tracknet_hybrid.py:9:The pipeline engine (healthcheck -> trajectory
-> windows -> AI handoff -> DB)
backend\pipeline\detectors\native_wasb_runner.py:57:# Single torch/cuda probe code, used by BOTH
healthcheck and the lazy device-resolution probe
backend\pipeline\detectors\native_wasb_runner.py:64:# healthcheck rejects anything else up front
so a typo can't reach argparse as rc=2 mid-run.
backend\pipeline\detectors\native_wasb_runner.py:135:                # Cached CUDA-availability probe
(used to resolve device="auto"). Set by healthcheck;
backend\pipeline\detectors\native_wasb_runner.py:136:                # lazily probed by run_predict if
healthcheck was skipped. None = not yet probed.
backend\pipeline\detectors\native_wasb_runner.py:163:                """CUDA availability, cached across
healthcheck + run_predict so we probe at most once."""
backend\pipeline\detectors\native_wasb_runner.py:194:    def healthcheck(self) -> Tuple[bool,
str]:
backend\pipeline\detectors\native_wasb_runner.py:228:                return False, "native torch
healthcheck timed out."
backend\pipeline\detectors\native_wasb_runner.py:304:                # Mirrors the healthcheck gate
for callers that skipped it (e.g. auto?cpu on a
backend\pipeline\segmenters\trajectory_hybrid.py:7:    healthcheck gate -> P2 trajectory ->
candidate windows (+ persistence)
backend\pipeline\segmenters\trajectory_hybrid.py:183:                # an unreadable video, an
unavailable detector (native-not-supported), the env healthcheck
backend\pipeline\segmenters\trajectory_hybrid.py:282:                # --- Phase 1: env healthcheck
(fail fast with a clear message) ---
backend\pipeline\segmenters\trajectory_hybrid.py:283:                ok, msg = runner.healthcheck()
backend\pipeline\detectors\stub_runner.py:84:    def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\segmenters\wasb_hybrid.py:11:The pipeline engine (healthcheck -> trajectory ->
windows -> AI handoff -> DB)
backend\pipeline\detectors\tracknet_runner.py:105:    def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\tracknet_runner.py:123:                return False, "WSL healthcheck
timed out."
backend\pipeline\detectors\wasb_runner.py:85:    def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\wasb_runner.py:97:                return False, "WSL healthcheck timed
out."

```

10. user (2026-07-07T04:11:21.698Z)

```

470     def _resolve_train_detector(args: argparse.Namespace, config: Any):
471         """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
472         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
473         stub=CPU mock). Returns the runner, or prints an error + returns None.
474
475         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
476         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
477         no shuttle detector and is rejected.
478         """
479         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
480
481         seg_cls = segmenter_registry.get(args.segmenter)
482         if not seg_cls:
483             print(

```

```

484         f"[worker] unknown segmenter: {args.segmenter!r} "
485         f"(have {list(segmenter_registry.list_available())})"
486     )
487     return None
488 seg = seg_cls(None, config)
489 if not isinstance(seg, TrajectoryHybridSegmenter):
490     print(
491         f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
492         f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
493     )
494     return None
495 try:
496     runner, _ = seg._resolve_runner(config.indexing)
497 except NotImplementedError as e:
498     # e.g. detector_impl='native' for a family without a native runner (tracknet, or
499     # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
500     print(f"[worker] detector not available: {e}")
501     return None
502 ok, msg = runner.healthcheck()
503 if not ok:
504     print(f"[worker] detector environment not ready: {msg}")
505     return None
506 print(f"[worker] detector ready ({args.segmenter}): {msg}")
507 return runner
508
509
510 def cmd_train(args: argparse.Namespace) -> int:
511     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
512     to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
513
514     Two trajectory sources (the train pipeline + harness are identical either way):
515     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
516         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
517     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
518         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
519         is the M3.5 "first real training" path; only the trajectory source flips.

```

11. user (2026-07-07T04:11:22.908Z)

```

270     ai_max_workers = idx_cfg.ai_max_workers
271     log_candidates = idx_cfg.log_yolo_candidates
272     store_candidates = idx_cfg.store_yolo_candidates
273
274     detection_params = {
275         "detector": self.name,
276         "velocity_thresh": velocity_thresh,
277         "merge_gap": merge_gap,
278         "min_action_window_duration": min_window_duration,
279         **runner_params,
280     }
281
282     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
283     ok, msg = runner.healthcheck()
284     if not ok:
285         logger.error("%s detector environment not ready: %s", label, msg)
286         # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
287         # failed/retryable instead of writing an empty-but-"successful" result.
288         return Failure(

```

```

289         message=f"{label} detector environment not ready: {msg}",
290         is_recoverable=True,
291     )
292     logger.info("%s detector env OK: %s", label, msg)
293
294     # --- Phase 2: trajectory -> candidate rally windows ---
295     # Trajectory detectors process the whole video in one pass (they carry
296     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
297     t_start = time.time()
298     out_dir = os.path.join(output_base(self.config), self.family)
299     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
300     if not csv_path:
301         logger.error("%s produced no trajectory; aborting.", label)
302         # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
303         # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
304         # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
305         # rather than a silent 0-segment "success" (the wf294 incident).
306         return Failure(
307             message=(
308                 f"{label} detector produced no trajectory (timed out or aborted) ? "
309                 "detector failure, not a 0-rally video."

```

12. user (2026-07-07T04:12:09.798Z)

Structured output provided successfully